CENTRO UNIVERSITÁRIO UNIRUY

DEPARATAMENTO DE TI

ÍCARO LIMA DE JESUS E RUAN SANTOS MÜLLER DE ALMEIDA

Documentação da Aplicação Web de Gestão de Frotas

Versão: Final

SALVADOR

2025

CENTRO UNIVERSITÁRIO UNIRUY

DEPARATAMENTO DE TI

ÍCARO LIMA DE JESUS E RUAN SANTOS MÜLLER DE ALMEIDA

Documentação da Aplicação Web de Gestão de Frotas

Versão: Final

Trabalho avaliativo semestral
apresentado a disciplina de Programação Orientada a Objetos em Java
pertencente ao Departamento de
TI do Centro Universitário Ruy Barbosa,
como parte dos requisitos para obtenção
de aprovação.
Professor: Heleno Cardoso

SALVADOR

2025

1. INTRODUÇÃO:

O presente projeto, consiste no desenvolvimento de uma aplicação web/mobile completa para a gestão e controle de frotas de veículos. O sistema foi concebido para atender às necessidades de empresas que precisam administrar de forma eficiente seus ativos móveis, motoristas, agendamentos de viagens, manutenções, abastecimentos e ocorrências.

O contexto da aplicação surge da complexidade logística enfrentada por transportadoras, empresas de entrega ou qualquer organização que dependa de veículos para suas operações. A falta de um sistema centralizado para gerenciar essas operações pode levar a ineficiências, custos elevados, falta de controle sobre a manutenção e dificuldades no rastreamento de viagens e despesas. A motivação para o desenvolvimento foi criar uma solução robusta, centralizada e de fácil utilização que resolvesse esses problemas, otimizando processos e fornecendo dados claros para a tomada de decisão. Abaixo segue os links dos repositórios com o código fonte do projeto:

Link da API: https://github.com/LJIcaro/gestaofrotas-api

Link da Interface: https://github.com/LJIcaro/gestaofrotas-ui

2. OBJETIVOS DA APLICAÇÃO:

A aplicação foi projetada com os seguintes objetivos e funcionalidades principais:

- **Gerenciar o cadastro completo de veículos:** Permitir o registro de veículos na frota, incluindo informações detalhadas como placa, marca, modelo, ano, chassi, RENAVAM e status (disponível, em viagem, em manutenção).

- **Gerenciar o cadastro de motoristas:** Manter um registro dos motoristas da empresa, com seus dados pessoais, número da CNH e status (ativo, inativo).

- **Realizar o agendamento de viagens:** Permitir que administradores criem, visualizem, editem e cancelem agendamentos, associando um motorista e um veículo a um destino e período específicos.

- **Controlar o ciclo de vida das viagens:** Implementar funcionalidades para iniciar e finalizar uma viagem, atualizando o status do veículo e do motorista em tempo real e registrando a quilometragem.

- **Registrar e gerenciar manutenções:** Permitir o cadastro de manutenções preventivas e corretivas, associando-as a um veículo, especificando o tipo de manutenção, custo e data.

- **Registrar e controlar abastecimentos:** Fornecer uma interface para registrar os abastecimentos, incluindo data, quantidade de litros, valor e quilometragem no momento do abastecimento, permitindo o cálculo de consumo.

- **Registrar ocorrências:** Permitir o registro de ocorrências durante as viagens, como acidentes ou multas, associando-as a um agendamento específico.

- **Autenticação e Segurança:** Garantir que apenas usuários autorizados (administradores) tenham acesso ao sistema através de uma tela de login segura.

- **Fornecer uma interface web responsiva:** Garantir que o sistema seja acessível e funcional tanto em desktops quanto em dispositivos móveis, facilitando o uso em campo.

3. FUNÇÕES/LISTA DE EVENTOS – RF / ENF:

Requisitos Funcionais (RF):

- **RF1:** O sistema deverá permitir a autenticação de usuários (administradores) através de login e senha.
- **RF2:** O sistema deverá permitir o cadastro, edição, visualização e exclusão de veículos da frota.
- **RF3:** O sistema deverá permitir o cadastro, edição, visualização e exclusão de motoristas.
- **RF4:** O sistema deverá permitir o cadastro, edição, visualização e cancelamento de agendamentos de viagens.
- **RF5:** O sistema deverá permitir que o administrador inicie uma viagem a partir de um agendamento, mudando o status do veículo para "EM_VIAGEM".
- **RF6:** O sistema deverá permitir que o administrador finalize uma viagem, registrando a quilometragem final e mudando o status do veículo para "DISPONIVEL".
- **RF7:** O sistema deverá permitir o registro de manutenções (preventivas e corretivas) para cada veículo.
- **RF8:** O sistema deverá permitir o registro de abastecimentos para cada veículo, incluindo custo e litragem.
- **RF9:** O sistema deverá permitir o registro de ocorrências (ex: multas, acidentes) vinculadas a um agendamento de viagem.
- **RF10:** O sistema deverá exibir listas paginadas de veículos, motoristas, agendamentos, manutenções e abastecimentos.

Requisitos Não Funcionais (RNF):

- **RNF1:** A aplicação deverá ser responsiva, adaptando-se a diferentes tamanhos de tela (desktops, tablets e smartphones).
- **RNF2:** O tempo de resposta para as principais operações do sistema (carregamento de listas, salvamento de formulários) não deve exceder 2 segundos em condições normais de rede.
- **RNF3:** A comunicação entre o frontend (Angular) e o backend (Java/Spring) deve ser segura, utilizando o protocolo HTTPS e autenticação baseada em tokens JWT (JSON Web Token).
- **RNF4:** O sistema deve ser desenvolvido utilizando a linguagem Java com o framework Spring Boot para o backend e TypeScript com o framework Angular para o frontend.
- **RNF5:** O sistema deve utilizar um banco de dados relacional (PostgreSQL) para a persistência dos dados.
- **RNF6:** A aplicação (backend e banco de dados) deve ser containerizada utilizando Docker para facilitar a implantação e garantir a consistência entre os ambientes de desenvolvimento e produção.

4. ESPECIFICAÇÃO DE PROGRAMAS:

a) Layout da Tela:

As telas da aplicação seguem um design limpo e funcional, focadas na usabilidade.

- **Tela de Login:** Uma tela simples contendo campos para "Usuário" e "Senha", e um botão "Entrar". Há também um link para a tela de cadastro de novos usuários.

- **Dashboard (Tela Principal):** Após o login, o usuário é direcionado para um painel principal que apresenta um menu lateral à esquerda e uma área de conteúdo principal. O menu contém links para as principais seções: "Veículos", "Motoristas", "Agendamentos", "Manutenções", "Abastecimentos". A área de conteúdo exibe cartões ou uma visão geral com atalhos para as funcionalidades.

- **Telas de Listagem (Veículos, Motoristas, etc.):** Apresentam os dados em formato de tabela, com colunas correspondentes aos principais atributos do registro. Cada linha possui botões de ação ("Editar", "Excluir"). Um botão "Novo" no topo da tela leva ao formulário de cadastro. A tabela possui paginação para lidar com grandes volumes de dados.

- **Telas de Formulário (Cadastro/Edição):** Contêm campos de entrada de texto, caixas de seleção (dropdowns) e seletores de data para preenchimento das informações da entidade (ex: formulário de cadastro

de veículo com campos para placa, marca, modelo, etc.). Possui os botões "Salvar" e "Cancelar".

-

b) Regras de Negócio:

- Um veículo não pode ser excluído se possuir agendamentos vinculados a ele.
- Um motorista não pode ser excluído se possuir agendamentos vinculados a ele.
- Não é possível criar um agendamento para um veículo que já esteja com status "EM_VIAGEM" ou "EM_MANUTENCAO".
- Não é possível criar um agendamento para um motorista que já esteja alocado em outra viagem no mesmo período.
- Ao iniciar uma viagem, o status do veículo é automaticamente alterado para "EM_VIAGEM".
- Ao finalizar uma viagem, a quilometragem final informada deve ser maior que a quilometragem inicial. Após a finalização, o status do veículo retorna para "DISPONIVEL".
- O sistema utiliza autenticação JWT. Um token é gerado no login e deve ser enviado no cabeçalho de cada requisição subsequente para acessar os recursos protegidos da API.

c) Entidades Envolvidas (Classes):

As principais classes do domínio da aplicação, localizadas no pacote com.pitfrota.gestaofrotas.model, são:

- **Veiculo:** Representa um veículo da frota. Atributos: id, placa, marca, modelo, chassi, renavam, anoFabricacao, anoModelo, status.
- **Motorista:** Representa um motorista. Atributos: id, nome, cpf, cnh, dataNascimento, status.
- **Agendamento:** Representa uma viagem agendada. Atributos: id, dataViagem, destino, quilometragemInicial, quilometragemFinal, status, e relacionamentos com Veiculo e Motorista.
- **Manutencao:** Representa um registro de manutenção. Atributos: id, data, tipoManutencao, custo, descricao, e relacionamento com Veiculo.
- **Abastecimento:** Representa um registro de abastecimento. Atributos: id, data, litros, valor, quilometragem, e relacionamento com Veiculo.
- **Ocorrencia: R**epresenta uma ocorrência durante uma viagem. Atributos: id, descricao, data, status, e relacionamento com Agendamento.
- **Administrador:** Representa o usuário do sistema. Atributos: id, nome, login, senha, status.

d) Tabelas:

As classes de entidade são mapeadas para as seguintes tabelas no banco de dados PostgreSQL, através da JPA/Hibernate:

- **veiculo:** Armazena os dados dos veículos.
- **motorista:** Armazena os dados dos motoristas.
- **agendamento:** Armazena os dados dos agendamentos de viagens.
- **manutencao:** Armazena o histórico de manutenções.
- **abastecimento:** Armazena o histórico de abastecimentos.
- **ocorrencia:** Armazena as ocorrências das viagens.
- **administrador:** Armazena os dados dos usuários do sistema.
- **historico_status_agendamento:** Tabela de log para registrar as mudanças de status de um agendamento.

5. DER OU DIAGRAMA DE CLASSE:

a) Artefato Gráfico (Descrição do Diagrama de Classe):

- A classe Agendamento possui uma relação de **Muitos-para-Um (@ManyToOne)** com Veiculo. Isso significa que muitos agendamentos podem estar associados a um único veículo, mas cada agendamento pertence a apenas um veículo.
- A classe Agendamento também possui uma relação de **Muitos-para-Um (@ManyToOne)** com Motorista. Muitos agendamentos podem ser realizados por um único motorista, mas cada agendamento é atribuído a um só motorista.
- A classe Manutencao possui uma relação de **Muitos-para-Um (@ManyToOne)** com Veiculo. Um veículo pode ter múltiplos registros de manutenção.
- A classe Abastecimento possui uma relação de **Muitos-para-Um (@ManyToOne)** com Veiculo. Um veículo pode ter múltiplos registros de abastecimento.
- A classe Ocorrencia possui uma relação de **Muitos-para-Um (@ManyToOne)** com Agendamento. Uma viagem agendada pode ter várias ocorrências registradas.
- A classe Veiculo possui uma relação de **Um-para-Muitos (@OneToMany)** com Agendamento, Manutencao e Abastecimento, representando o lado inverso das relações descritas acima.
- A classe Motorista possui uma relação de **Um-para-Muitos (@OneToMany)** com Agendamento.
- A classe Agendamento possui uma relação de **Um-para-Muitos (@OneToMany)** com Ocorrencia.

b) Dicionário de Dados (DD):

Abaixo está um dicionário de dados para os atributos mais importantes das principais tabelas/classes.

- **Tabela veiculo:**
  - id (BIGINT, PK): Identificador único do veículo.
  - placa (VARCHAR): Placa do veículo (ex: "ABC-1234").
  - marca (VARCHAR): Fabricante do veículo (ex: "Volkswagen").
  - modelo (VARCHAR): Modelo do veículo (ex: "Gol").
  - status (VARCHAR): Status atual do veículo ("DISPONIVEL", "EM_VIAGEM", "EM_MANUTENCAO").

- **Tabela motorista:**
  - id (BIGINT, PK): Identificador único do motorista.
  - nome (VARCHAR): Nome completo do motorista.
  - cpf (VARCHAR): CPF do motorista.
  - cnh (VARCHAR): Número da Carteira Nacional de Habilitação.
  - status (VARCHAR): Status do motorista ("ATIVO", "INATIVO").

- **Tabela agendamento:**
  - id (BIGINT, PK): Identificador único do agendamento.
  - data_viagem (TIMESTAMP): Data e hora de início da viagem.
  - destino (VARCHAR): Descrição do destino da viagem.
  - km_inicial (BIGINT): Quilometragem do veículo no início da viagem.

- km_final (BIGINT): Quilometragem do veículo no fim da viagem.
- status (VARCHAR): Status do agendamento ("PENDENTE", "EM_ANDAMENTO", "FINALIZADO", "CANCELADO").
- veiculo_id (BIGINT, FK): Chave estrangeira referenciando tb_veiculo.
- motorista_id (BIGINT, FK): Chave estrangeira referenciando tb_motorista.

6. APLICAÇÃO WEB/MOBILE:

A aplicação web, hospedada em https://pitfrotas.online/, é a interface principal para interação com o sistema de gestão de frotas.

a) Menu/Submenu:

O sistema possui um menu de navegação principal, localizado na lateral esquerda da tela, que se expande ou se recolhe. As opções do menu são:

- **Dashboard:** Tela inicial com visão geral.
- **Veículos:** Leva à tela de listagem de todos os veículos da frota.
- **Motoristas:** Leva à tela de listagem de todos os motoristas.
- **Agendamentos:** Leva à tela de listagem de todas as viagens agendadas.
- **Manutenções:** Leva à tela de listagem do histórico de manutenções.
- **Abastecimentos:** Leva à tela de listagem do histórico de abastecimentos.
- **Sair:** Realiza o logout do sistema.

Não há submenus; a navegação é direta para cada funcionalidade principal.

b) Telas Funcionais:

- **Tela de Login:** Responsável pela autenticação do usuário.
- **Tela de Listagem de Veículos:** Exibe a frota em uma tabela com Placa, Modelo, Marca, Status e botões de ação. Permite iniciar a criação de um novo veículo.
- **Tela de Cadastro de Veículo:** Formulário para inserir ou editar dados de um veículo.
- **Tela de Listagem de Agendamentos:** Mostra as viagens com Destino, Veículo, Motorista, Data e Status. Ações como "Iniciar Viagem", "Finalizar Viagem" e "Cancelar" estão disponíveis conforme o status do agendamento.

- **Tela de Cadastro de Agendamento:** Formulário onde o administrador seleciona um veículo e um motorista (ambos disponíveis) e preenche os detalhes da viagem.

c) Telas de Diálogo:

O sistema utiliza diálogos modais para interações importantes com o usuário, melhorando a experiência e prevenindo ações acidentais.

- **Confirmação de Exclusão:** Ao clicar no botão "Excluir" em qualquer registro (veículo, motorista, etc.), um modal de confirmação surge com a pergunta "Você tem certeza que deseja excluir este item?" e os botões "Sim" e "Não".

- **Mensagens de Erro:** Caso ocorra um erro de validação (ex: tentar salvar um formulário com campos obrigatórios em branco) ou um erro de comunicação com o servidor, uma notificação (toast) aparece no canto da tela com uma mensagem descritiva, como "Erro ao salvar veículo: verifique os campos" ou "Placa já cadastrada".

- **Mensagens de Sucesso:** Após uma operação bem-sucedida (ex: salvar um novo motorista), uma notificação de sucesso é exibida, como "Motorista salvo com sucesso!".

- **Diálogo para Finalizar Viagem:** Ao clicar em "Finalizar Viagem", um modal específico é aberto, solicitando que o administrador insira a quilometragem final do veículo antes de confirmar a ação.

d) Layout Relatórios:

Embora a versão atual da aplicação se concentre nas funcionalidades operacionais (CRUDs e controle de status), a estrutura de dados permite a geração de relatórios gerenciais cruciais. Um layout de relatório típico seria:

- **Relatório de Consumo por Veículo:**
    - **Layout:** Uma tela com filtros por período (data de início e fim) e por veículo (placa).
    - **Apresentação:** Os dados seriam exibidos em uma tabela contendo as colunas: Data do Abastecimento, Litros, Valor (R$), KM Percorrida desde o último abastecimento, Consumo (KM/L).
    - **Gráficos:** Um gráfico de linhas poderia ser exibido abaixo da tabela, mostrando a evolução do consumo (KM/L) do veículo ao longo do tempo, ajudando a identificar problemas mecânicos.

- **Relatório de Custos de Manutenção:**
    - **Layout:** Filtros por período e por tipo de manutenção (preventiva/corretiva).
    - **Apresentação:** Uma tabela com Veículo (Placa), Data da Manutenção, Tipo, Descrição do Serviço, Custo (R$). Um totalizador no final da página mostraria o custo total no período.
    - **Gráficos:** Um gráfico de pizza poderia mostrar a distribuição de custos entre os tipos de manutenção.